National Telecommunication Institute

IT INDUSTRY DEVELOPMENT AGENCY

Systolic Array-Based Accelerator for CNNs

Prepared for:

Eng: Mohammed Salah Aboshosha

Team Members Contributions

Name	Contribution
Adel Wael	Test Benches
Mohamed Ramadan Mostafa	Systolic Array and Delay
Kirollos Ibrahim	Processing Element
Youssef Michel	Python Code Program
Mohamed Atif	Image to Column, FIFO and Control

Introduction

- Convolutional Neural Networks (CNNs) are widely used in image processing and computer vision.
- CNNs require a large number of multiplication and accumulation operations, resulting in high computational demands.
- Our project focuses on developing a hardware accelerator to execute these operations efficiently.
- The accelerator uses a Systolic Array of Processing Elements (PEs) to perform computations in parallel.
- Convolution operations are converted into matrix multiplication, allowing the systolic array to achieve high computational throughput.

Software Bottlenecks

- **Loop & Instruction Overhead**

  Normal CPUs use software loops to perform matrix multiplication. This requires many instructions to be fetched, decoded, and executed, which adds extra time and processing overhead.

- **The Memory Wall**

  Traditional CPUs need to get data from main memory for each Multiply-Accumulate (MAC) operation. Because memory is slower than the processor, this can cause delays and limit performance.

- **Power Inefficiency**

  Moving data between memory and the CPU uses a lot of power. In many cases, moving the data consumes more energy than performing the actual calculations.

Hardware Acceleration

- **Spatial Processing Grid**

  Replaces sequential instruction execution with a parallel 2D Processing Element (PE) grid.

- **Local Data Reuse**

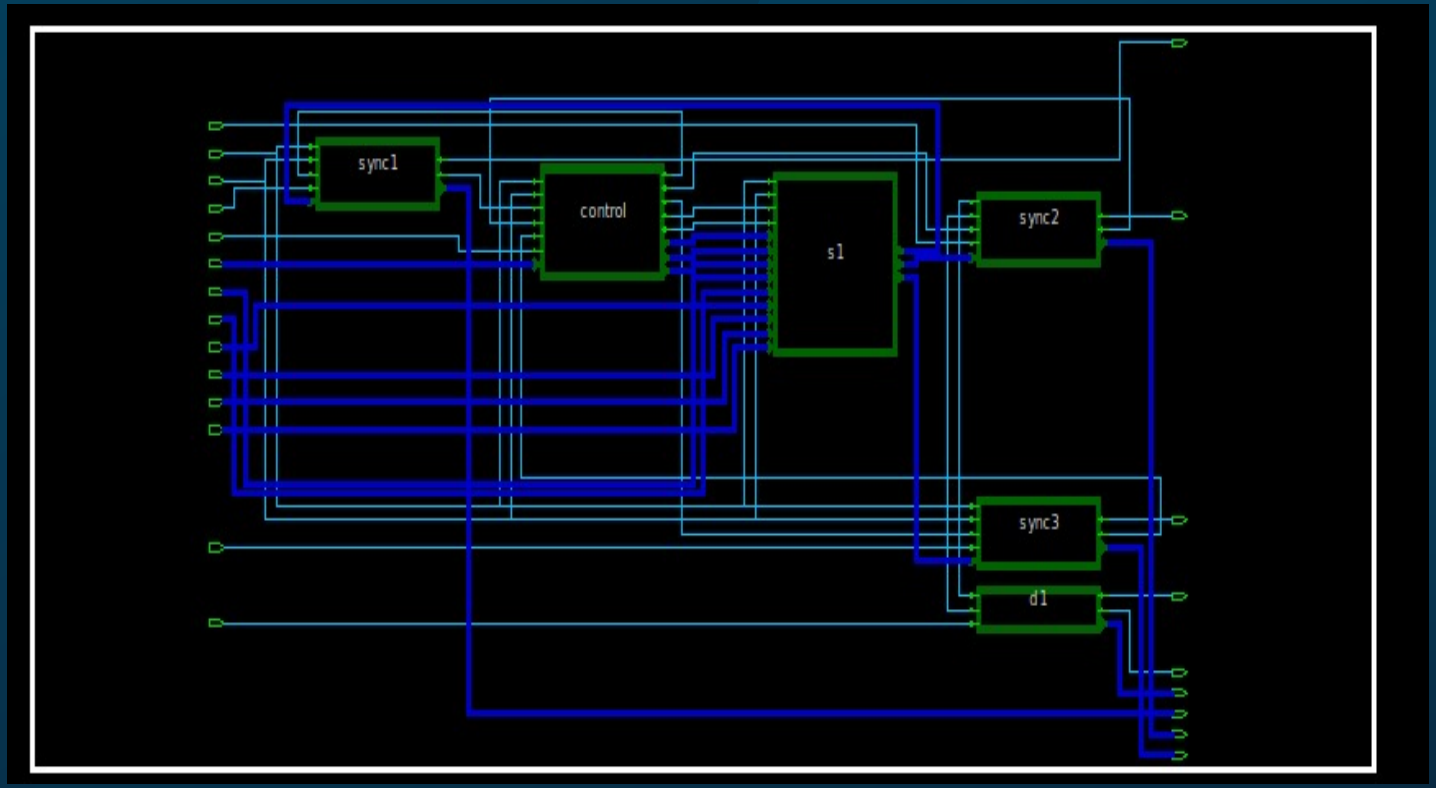
  Input operands stream across adjacent PEs,
reusing data horizontally and vertically without repeated memory reads.

- **Power Inefficiency**

  Synchronous FIFOs decouple high-speed hardware matrix processing from lower-speed system/bus reading domains.

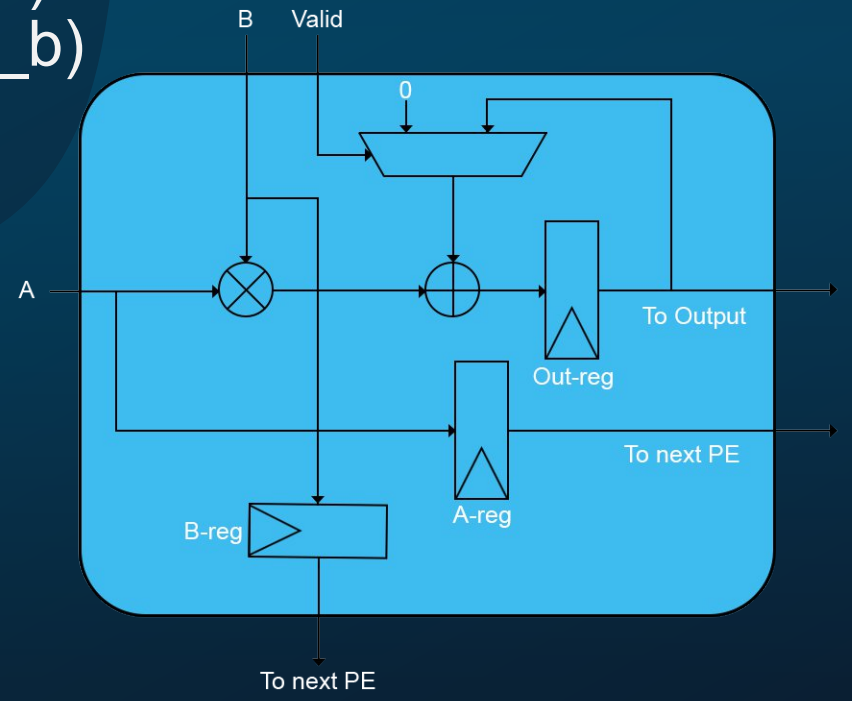
Top-Level Architecture

- **Top Module:** systolic_array_ai
  - Systolic Array Engine
    (3x3 Matrix Core)
    Module Name: systolic_array
  - Control System Unit
    (FSM and Control Signals)
    Module Name: control_system1
  - Synchronous FIFO Array
    (3x Output Channels)
    Module Name: sync_fifo



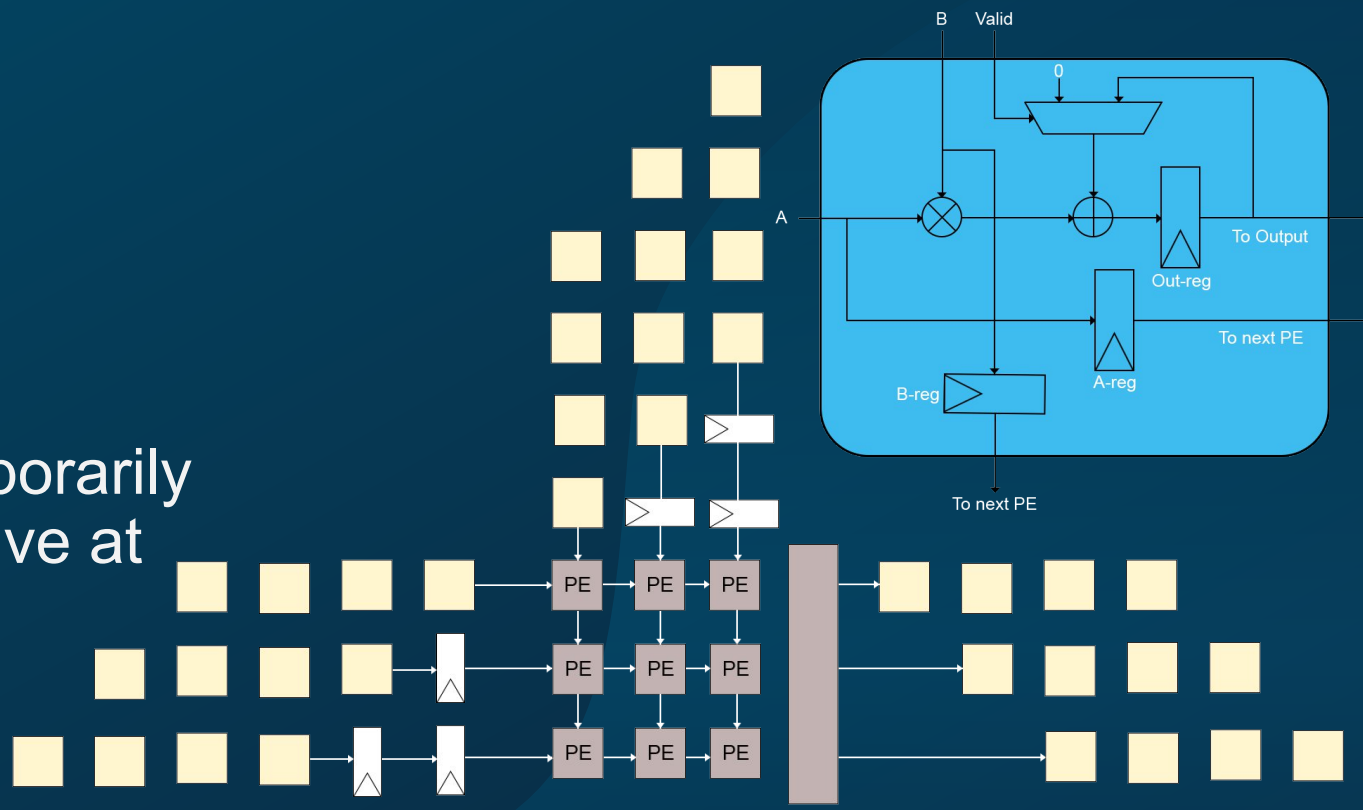
Processing Element (PE)

- Module Name: PE
- Function: serves as the fundamental unit performing MAC operations across the matrix grid.
- Registers: registers input operands (in_PE_a, in_PE_b) to forward them to neighboring right and bottom PEs (out_PE_a, out_PE_b) on every clock edge.
- Accumulation Logic:
  - When in_PE_valid = 1: accumulates product into register **out_PE_acc = out_PE_acc + (a * b)**
  - When in_PE_valid = 0: overwrites accumulation with fresh product **out_PE_acc = a * b**



Delay Skewing

- Module Name: DELAY
- Function: align matrix inputs temporarily so row and column elements arrive at the correct PE simultaneously.
- Configuration:
  - Row 0 / Column 0: direct input (0 cycle delay).
  - Row 1 / Column 1: shifted be 1 clock cycle (DELAY_CYCLE = 1).
  - Row 2 / Column 2: shifted be 2 clock cycle (DELAY_CYCLE = 2).



FSM Control System

- Module Name: control_system1
- Control Output Signals:
  - Valid Flag (out_cs_valid): asserts high when computation is active and no FIFO buffer is in a full state (!in_cs_full1 && !in_cs_full2 && !in_cs_full3).
  - Multiplexer Controls (out_cs_ctrl1_3): dynamically routes individual PE accumulator outputs (out[r][c]) to output buses based on cycle position.
  - FIFO Write Enables (out_cs_winc1_3): generates staged write-increment pulses to push completed calculations into output buffers safely.

Synchronous FIFO

- Module Name: sync_fifo
- Function: small memory queue that moves data safely between stages.
- We used Synchronous FIFO instead of Asynchronous FIFO because of speed, simple and small design.
- Flow Control and Safety:
  - Full Generation: asserts full signal back to control_system1 to automatically stall accelerator writes when buffers fill up.
  - Empty Generation: asserts empty signal to external software read-drivers when output data is unavailable.

Thank You!